Chủ đề 1: Tổng quan về lập trình cho thiết bị di động (số tiết: 5 tiết)

Nội dung chính

1.1. Lập trình cho thiết bị di động: Đặc điểm, phương pháp và các công cụ lập trình

1.2. Lập trình cơ bản với ngôn ngữ Kotlin

Mục tiêu cần đạt được

Hiểu các đặc điểm và phương pháp lập trình cho thiết bị di động, cùng với các yếu tố quan trọng như tối ưu hóa hiệu suất, giao diện người dùng (UI), và tương thích với thiết bị đa dạng.

Biết về các công cụ lập trình phổ biến như Android Studio, SDK Android, và ngôn ngữ Kotlin, cùng cách sử dụng để phát triển ứng dụng Android.

Vận dụng kiến thức về công cụ lập trình và ngôn ngữ Kotlin để phát triển ứng dụng Android cơ bản, bao gồm tạo giao diện, xử lý sự kiện và làm việc với các API.

1.1. Lập trình cho thiết bị di động: Đặc điểm, phương pháp và các công cụ lập trình (2 tiết)

1.1.1. Đặc điểm khi lập trình cho thiết bị di động

Khi lập trình cho thiết bị di động nói chung, lập trình viên phải đối mặt với nhiều vấn đề kỹ thuật và thiết kế khác nhau. Các yếu tố cần quan tâm bao gồm:

- **Tính đa dạng của thiết bị**: Thiết bị di động có sự đa dạng lớn về kích thước màn hình, độ phân giải, bộ nhớ và hiệu suất phần cứng. Lập trình viên cần đảm bảo rằng ứng dụng có khả năng hoạt động tốt trên nhiều thiết bị khác nhau, bao gồm cả điện thoại, máy tính bảng và đồng hồ thông minh. Điều này yêu cầu ứng dụng phải có giao diện linh hoạt và tương thích với nhiều loại màn hình.
- **Tối ưu hóa hiệu suất và tài nguyên**: Do hạn chế về tài nguyên như bộ nhớ và pin, ứng dụng di động cần được tối ưu hóa để chạy tốt và không gây lãng phí tài nguyên. Các chiến lược phổ biến bao gồm quản lý bộ nhớ hiệu quả, sử dụng các API tiết kiệm năng lượng, và tối ưu hóa việc lưu trữ và truy xuất dữ liệu cục bộ.

- **Tương tác người dùng và trải nghiệm giao diện**: Giao diện người dùng (UI) trên thiết bị di động đòi hỏi sự trực quan và dễ sử dụng. Người dùng thường tương tác với ứng dụng thông qua cảm ứng, do đó cần thiết kế các giao diện dễ hiểu, tối giản và hỗ trợ các thao tác chạm, kéo thả, và vuốt.
- **Kết nối mạng và dữ liệu**: Ứng dụng di động thường cần kết nối mạng để lấy dữ liệu hoặc đồng bộ hóa với các dịch vụ đám mây. Lập trình viên phải đảm bảo quản lý kết nối mạng hiệu quả, xử lý các trạng thái không có kết nối, và tối ưu hóa lượng dữ liệu sử dụng để tiết kiệm băng thông và pin.
- **Bảo mật**: Vì thiết bị di động thường lưu trữ thông tin cá nhân nhạy cảm, bảo mật là yếu tố quan trọng trong lập trình di động. Các vấn đề bảo mật bao gồm mã hóa dữ liệu, xác thực người dùng và bảo vệ chống lại các cuộc tấn công như mã độc hoặc xâm nhập dữ liệu.
- **Quản lý tài nguyên và quyền truy cập**: Ứng dụng di động phải xử lý quyền truy cập các tài nguyên hệ thống như máy ảnh, vị trí, và danh bạ một cách hợp lý. Việc yêu cầu quyền truy cập không cần thiết có thể làm giảm trải nghiệm người dùng và gây ra các vấn đề bảo mật.

Những yếu tố trên cho thấy rằng lập trình di động đòi hỏi sự hiểu biết sâu rộng về cách thức hoạt động của thiết bị di động, từ quản lý tài nguyên đến tối ưu hóa giao diện và bảo mật thông tin người dùng.

1.1.2. Các phương pháp lập trình cho thiết bị di động

Các phương pháp lập trình cho thiết bị di động thường liên quan đến việc chọn cách tiếp cận phù hợp để xây dựng ứng dụng trên các nền tảng khác nhau, như Android, iOS hoặc các nền tảng đa nền tảng. Dưới đây là những phương pháp chính:

Phát triển gốc (Native Development)

Đây là phương pháp truyền thống, các ứng dụng được phát triển riêng cho từng hệ điều hành bằng các ngôn ngữ lập trình gốc:

- Android: Sử dụng Java hoặc Kotlin trong môi trường Android Studio.
- iOS: Sử dụng Swift hoặc Objective-C trong Xcode.

Ưu điểm của phát triển gốc là ứng dụng tận dụng được tối đa các tính năng của hệ điều hành và phần cứng thiết bị. Tuy nhiên, việc phát triển riêng cho từng nền tảng có thể tốn kém thời gian và tài nguyên.

Phát triển đa nền tảng (Cross-Platform Development)

Phát triển đa nền tảng giúp lập trình viên xây dựng một ứng dụng có thể chạy trên nhiều hệ điều hành bằng cách sử dụng một mã nguồn duy nhất. Các công nghệ phổ biến bao gồm:

- React Native: Sử dụng JavaScript và React, cho phép chia sẻ mã giữa Android và iOS.
- Flutter: Một framework của Google sử dụng ngôn ngữ Dart, hỗ trợ cả Android và iOS.
- Xamarin: Sử dụng C# và cung cấp khả năng phát triển đa nền tảng thông qua .NET.

Ưu điểm của phương pháp này là tiết kiệm thời gian phát triển, nhưng nhược điểm có thể là không tối ưu hiệu suất hoặc không hỗ trợ đầy đủ các tính năng đặc thù của từng nền tảng.

Phát triển ứng dụng web di động (Mobile Web Development)

Ứng dụng web di động là các trang web hoặc ứng dụng chạy trên trình duyệt di động nhưng được tối ưu hóa cho giao diện người dùng trên các thiết bị di động. Các công nghệ được sử dụng bao gồm:

- HTML5, CSS3, JavaScript: Làm nền tảng cho các ứng dụng web di động.
- Progressive Web Apps (PWA): Là một phương pháp cho phép các ứng dụng web hoạt động tương tự như ứng dụng gốc với khả năng hoạt động ngoại tuyến và đẩy thông báo.

Phương pháp này tiết kiệm thời gian và chi phí, nhưng không thể truy cập vào tất cả các chức năng phần cứng của thiết bị như ứng dụng gốc.

Phát triển ứng dụng lai (Hybrid Development)

Ứng dụng lai là sự kết hợp giữa phát triển gốc và web. Chúng được phát triển bằng công nghệ web (HTML, CSS, JavaScript) và sau đó được "gói" trong một container gốc như WebView trên Android hoặc WKWebView trên iOS. Các công cụ phổ biến gồm:

- Apache Cordova: Cho phép các ứng dụng web hoạt động như ứng dụng gốc.
- Ionic Framework: Cung cấp giao diện ứng dụng di động với các thành phần UI gốc.

Mặc dù phương pháp này đơn giản hóa quá trình phát triển đa nền tảng, nhưng hiệu suất và trải nghiệm người dùng không bằng ứng dụng gốc.

Nguyên tắc chung trong thiết kế giao diện di động:

- Bố cục và cấu trúc: Thiết kế giao diện ứng dụng di động cần ưu tiên tính đơn giản và dễ sử dụng. Điều này có nghĩa là nội dung cần được sắp xếp hợp lý để hướng dẫn người dùng thực hiện các nhiệm vụ một cách hiệu quả. Các bố cục thường sử dụng lưới hoặc các khung linh hoạt để thích ứng với các kích thước màn hình khác nhau.
- Chữ viết và màu sắc: Việc sử dụng kiểu chữ hợp lý giúp cải thiện khả năng đọc, trong khi một bảng màu phù hợp có thể gợi lên các cảm xúc cụ thể và định hướng hành vi của người dùng. Font chữ cần phải phản hồi tốt, cùng với màu sắc có độ tương phản cao để đảm bảo dễ đọc cho mọi đối tượng người dùng.
- Tính tương tác và phản hồi: Các yếu tố UI như nút bấm, cử chỉ vuốt, và các thao tác cảm ứng nên phản hồi ngay lập tức với các lệnh của người dùng, cung cấp phản hồi qua hình ảnh hoặc âm thanh để đảm bảo người dùng cảm thấy họ đang kiểm soát ứng dụng.
- Tính nhất quán: Đảm bảo sự nhất quán trong thiết kế giao diện trên toàn bộ ứng dụng, bao gồm việc sử dụng phông chữ, màu sắc, các nút, và mô hình điều hướng giống nhau. Điều này sẽ giảm bớt sự học hỏi của người dùng và tăng tính tiện dụng.
- Kiểm tra và lặp lại: Thiết kế UI cho các thiết bị di động cần có sự kiểm tra và phản hồi từ người dùng. Việc kiểm tra liên tục với người dùng thực tế giúp nhận ra các vấn đề và khu vực cần cải thiện, từ đó giúp giao diện thân thiện hơn.

Thiết kế UI đặc trưng cho Android:

Android UI tuân theo các nguyên tắc của Material Design, nhấn mạnh vào việc tạo ra trải nghiệm đồng nhất trên các nền tảng khác nhau bằng cách sử dụng các thành

phần như thẻ (cards), nút hành động nổi (FAB), và lưới phản hồi. Các yếu tố chính trong Android bao gồm:

- Jetpack Compose: Đây là bộ công cụ hiện đại giúp đơn giản hóa việc phát triển UI trong Android, cho phép nhà phát triển tạo giao diện động một cách khai báo. Compose giúp giảm bớt mã thừa, dễ dàng quản lý các giao diện phức tạp.
- Thành phần và chủ đề (Themes): Android cung cấp các thành phần UI được xây dựng sẵn (như nút bấm, hộp thoại, và menu) theo các hướng dẫn của Material Design. Việc sử dụng các thành phần này giúp đảm bảo giao diện đồng nhất trong ứng dụng. Chủ đề (themes) cho phép tùy chỉnh giao diện để phù hợp với thương hiệu.
- Bố cục thích ứng: Vì Android chạy trên nhiều loại thiết bị (điện thoại, máy tính bảng, TV), các nhà thiết kế cần cân nhắc đến kích thước và độ phân giải màn hình khác nhau. Android hỗ trợ bố cục linh hoạt, điều chỉnh dựa trên kích thước và hướng của màn hình, đảm bảo trải nghiệm mượt mà cho tất cả người dùng.
- Mô hình điều hướng: Các thành phần điều hướng phổ biến trong Android bao gồm thanh điều hướng dưới cùng (bottom navigation bars), menu ngăn kéo (drawer menus), và các tab, tất cả đều được thiết kế để tăng trải nghiệm người dùng và tính khả dụng. Các thành phần điều hướng trong Jetpack giúp tạo ra các luồng điều hướng linh hoạt và mạnh mẽ.

Bằng cách tích hợp các phương pháp này vào phát triển giao diện người dùng di động, dù là cho Android hay các nền tảng khác, sẽ đảm bảo rằng ứng dụng của mình không chỉ có giao diện đẹp mà còn cung cấp trải nghiệm mượt mà, thân thiện với người dùng.

1.1.3. Các ngôn ngữ phổ biến trong lập trình di động

Dưới đây là danh sách các ngôn ngữ lập trình phổ biến trong phát triển ứng dụng di động:

- Java: Ngôn ngữ chính cho phát triển Android.
- Kotlin: Ngôn ngữ chính thức cho Android, hiện đại và an toàn.
- Swift: Ngôn ngữ chính cho phát triển ứng dụng iOS.
- Objective-C: Ngôn ngữ cũ cho iOS, vẫn còn nhiều ứng dụng sử dụng.
- Dart: Ngôn ngữ cho phát triển đa nền tảng với Flutter.

- JavaScript: Sử dụng trong React Native để phát triển ứng dụng di động.
- TypeScript: Mở rộng của JavaScript, cung cấp kiểu tĩnh.
- C#: Ngôn ngữ cho phát triển đa nền tảng với Xamarin.
- PHP: Thường dùng cho backend, cung cấp API cho ứng dụng di động.
- Rust: Ngôn ngữ an toàn và hiệu suất cao, ngày càng được sử dụng trong di động.

1.1.4. Các công cụ phổ biến trong lập trình di động

Dưới đây là một số công cụ hỗ trợ lập trình cho thiết bị di động giúp nhà phát triển thiết kế, phát triển và kiểm thử ứng dụng một cách hiệu quả:

Figma

- Nền tảng: Đa nền tảng
- Chức năng: Công cụ thiết kế UI/UX trực tuyến với khả năng cộng tác thời gian thực. Figma cho phép nhà thiết kế và nhà phát triển cùng làm việc trên một giao diện, tạo prototype và chuyển thiết kế thành mã dễ dàng thông qua các plugin tích hợp như Zeplin, Flutter.

Android Studio

- Nền tảng: Android
- Chức năng: IDE chính thức cho phát triển Android, hỗ trợ ngôn ngữ Kotlin, Java, C++, cung cấp các tính năng thiết kế giao diện, giả lập, và quản lý phiên bản mã nguồn.

Xcode

- Nền tảng: iOS (Apple)
- Chức năng: IDE chính thức để phát triển ứng dụng iOS, hỗ trợ Swift và Objective-C, cung cấp các công cụ thiết kế giao diện và kiểm thử ứng dụng.

Flutter

- Nền tảng: Android, iOS, Web, Desktop

- Chức năng: SDK đa nền tảng từ Google, sử dụng ngôn ngữ Dart, giúp phát triển ứng dụng với một mã nguồn cho nhiều nền tảng khác nhau.

React Native

- Nền tảng: Android, iOS
- Chức năng: Framework từ Facebook cho phép phát triển ứng dụng di động bằng JavaScript và React, với khả năng chia sẻ mã giữa các nền tảng.

Firebase

- Nền tảng: Android, iOS, Web
- Chức năng: Bộ công cụ từ Google hỗ trợ cơ sở dữ liệu, xác thực người dùng, phân tích, lưu trữ đám mây và các tính năng backend cho ứng dụng di động.

Expo

- Nền tảng: Android, iOS
- Chức năng: Framework hỗ trợ phát triển ứng dụng nhanh với React Native, cung cấp công cụ debug, triển khai dễ dàng và hỗ trợ tính năng hot-reload.

GitHub/GitLab

- Nền tảng: Đa nền tảng
- Chức năng: Công cụ quản lý mã nguồn, giúp dễ dàng chia sẻ, quản lý phiên bản và cộng tác phát triển ứng dụng.

Postman

- Nền tảng: Đa nền tảng
- Chức năng: Công cụ kiểm thử API, giúp nhà phát triển kiểm tra và tương tác với API backend.

Jira

- Nền tảng: Đa nền tảng
- Chức năng: Công cụ quản lý dự án, giúp theo dõi tiến độ, lỗi và phối hợp giữa các thành viên trong nhóm phát triển.

Zeplin

- Nền tảng: Đa nền tảng

Chức năng: Cầu nối giữa nhà thiết kế và nhà phát triển, giúp chuyển đổi thiết kế từ Figma, Sketch sang mã CSS, Android XML hoặc Swift dễ dàng.

1.1.5. Cài đặt IDE, máy ảo cho lập trình Android

1.1.5.1. Android Studio

Android Studio là môi trường phát triển tích hợp (IDE) chính thức dùng để phát triển ứng dụng Android, do Google xây dựng và phân phối. IDE chứa các công cụ cho phép nhà phát triển phần mềm thiết kế, dựng, chạy và kiểm thử phần mềm, trong trường hợp này là các ứng dụng dành cho nền tảng Android. Android Studio dùng IntelliJ IDEA làm nền tảng và có plugin Android được cài đặt sẵn cùng với một số tuỳ chọn sửa đổi dành riêng cho nền tảng Android.

Yêu cầu về hệ thống của Android Studio

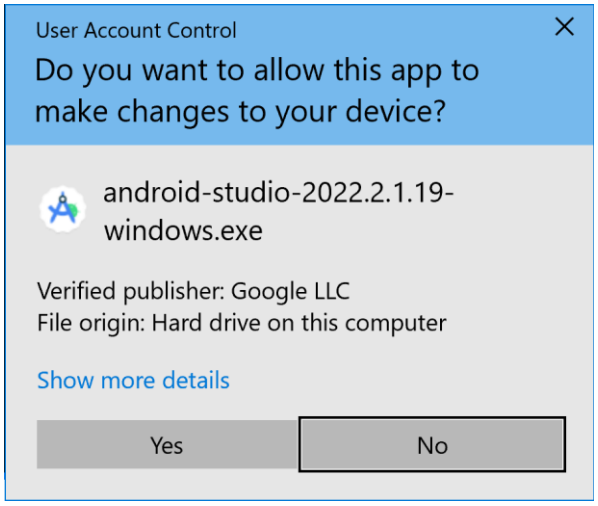
- Microsoft® Windows® 8/10/11 64 bit
- Kiến trúc CPU x86_64; Intel Core thế hệ 2 trở lên hoặc CPU AMD có hỗ trợ Trình điều khiển ảo hoá của Windows
- RAM 8 GB trở lên
- Độ phân giải màn hình tối thiểu là 1280 x 800

Tải Android Studio xuống

Tải Android Studio từ địa chỉ: https://developer.android.com/studio?hl=vi#get-android-studio, Và làm theo hướng dẫn trên trang.

Cài đặt Android Studio trên Windows

Mở thư mục nơi đã tải và lưu tệp cài đặt Android Studio. Nhấp đúp vào tệp đã tải xuống và tiến hành cài đặt.



Hình 1.1: Cài đặt Android Studio

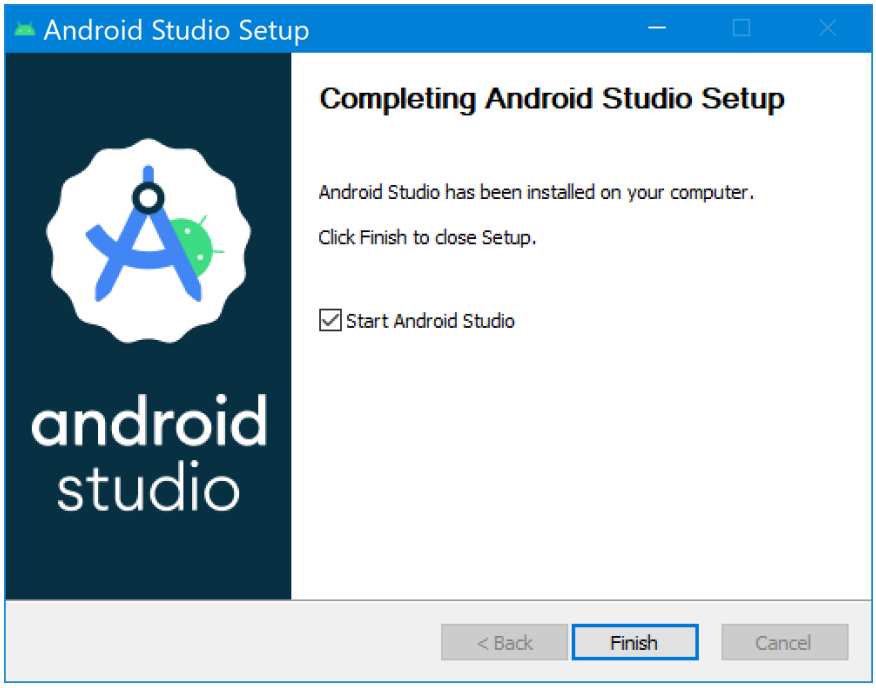
Hộp thoại **Welcome to Android Studio Setup** (Chào mừng đến với phần Thiết lập Android Studio) sẽ hiện ra.

Hình 1.2: Màn hình Wellcome

Nhấp vào **Next** (Tiếp theo) để bắt đầu cài đặt.

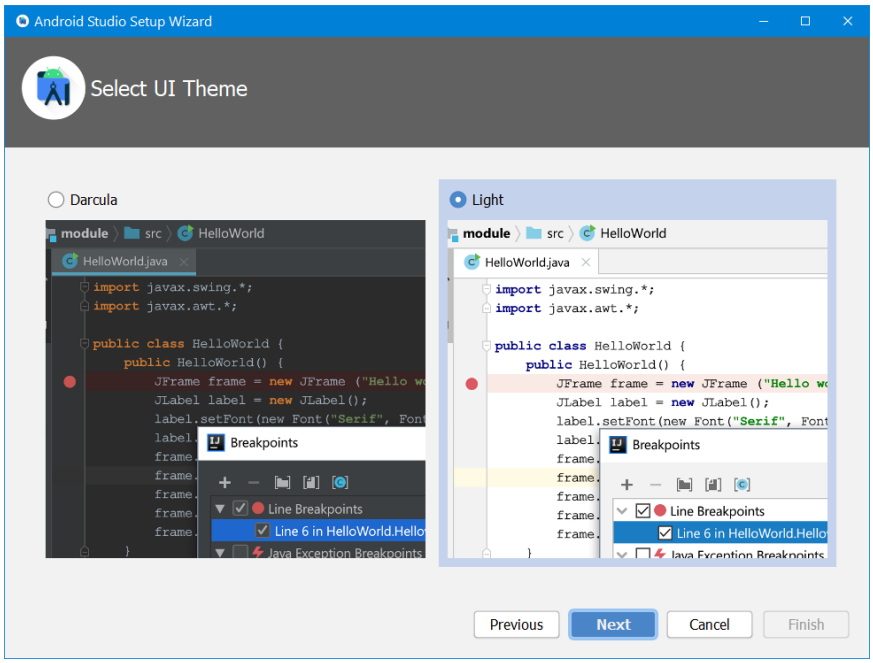
Chấp nhận các chế độ cài đặt mặc định cho mọi bước.

Nhấp vào **Finish** (Hoàn tất) khi quá trình cài đặt hoàn tất để chạy Android Studio.



Hình 1.3: Cài đặt hoàn tất

Chọn lựa chọn ưu tiên về giao diện sáng hoặc tối khi Android Studio khởi chạy lần đầu tiên.

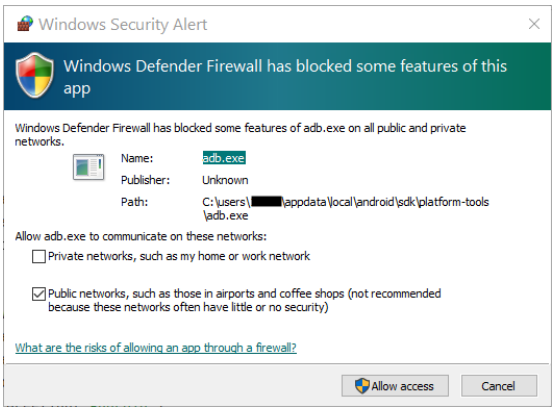


Hình 1.4: Lựa chọn giao diện

Trong quá trình cài đặt, trình hướng dẫn thiết lập sẽ tải xuống và cài đặt các thành phần cũng như công cụ bổ sung cần thiết để phát triển ứng dụng Android. Quá trình này có thể mất chút thời gian tuỳ thuộc vào tốc độ Internet.

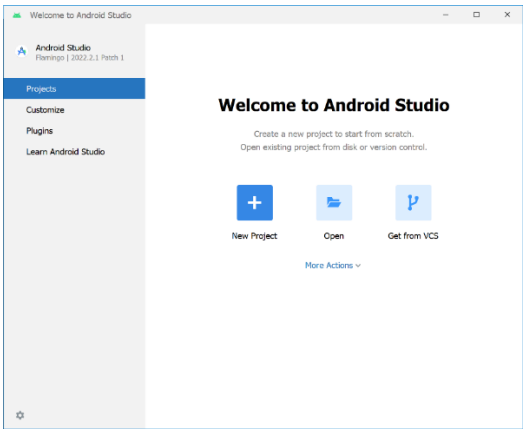
Hình 1.5: Cho phép cài đặt thêm

Cửa sổ **Windows Security Alert** (Cảnh báo bảo mật của Windows) có thể hiện lên. Nhấp vào **Allow Access** (Cho phép truy cập) để tiếp tục cài đặt, nếu cần.



Hình 1.6: Cửa sổ cảnh báo của Window

Khi quá trình tải xuống và cài đặt hoàn tất, hãy nhấp vào **Finish** (Hoàn tất). Cửa sổ **Welcome to Android Studio** (Chào mừng đến với Android Studio) sẽ hiện ra:



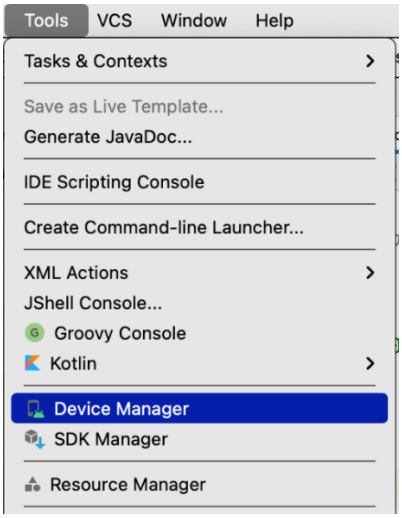
Hình 1.7: Cài đặt thêm các thành phần hoàn tất

1.1.5.2. Thiết bị mô phỏng

Để chạy demo các ứng dụng Android, người lập trình có thể sử dụng thiết bị ảo hoặc là thiết bị thật, có rất nhiều công cụ cho phép tạo ra thiết bị ảo Android, dưới đây bài giảng sẽ hướng dẫn sử dụng thiết bị ảo – AVD trong Android Studio và cách kết nối thiết bị thật với Android Studio để chạy mô phỏng

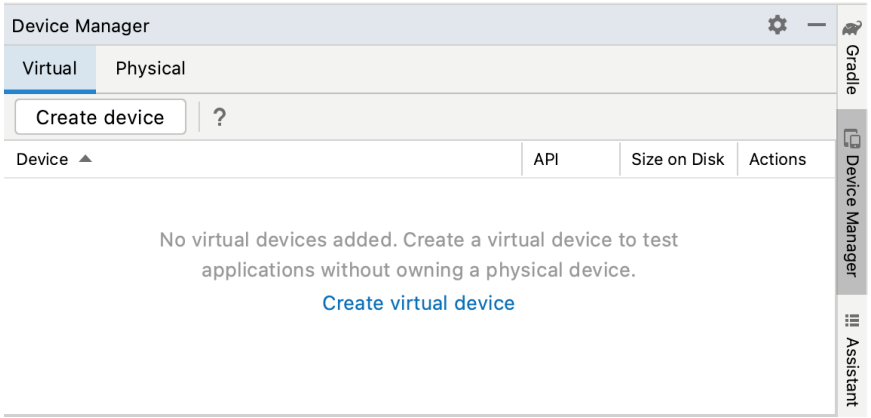
a. Thiết bị ảo – AVD

Để chạy một ứng dụng Android trong trình mô phỏng trên máy tính, ta có thể tạo một thiết bị ảo. Trong Android Studio, hãy chọn **Tools > Device Manager** (Công cụ > Trình quản lý thiết bị).



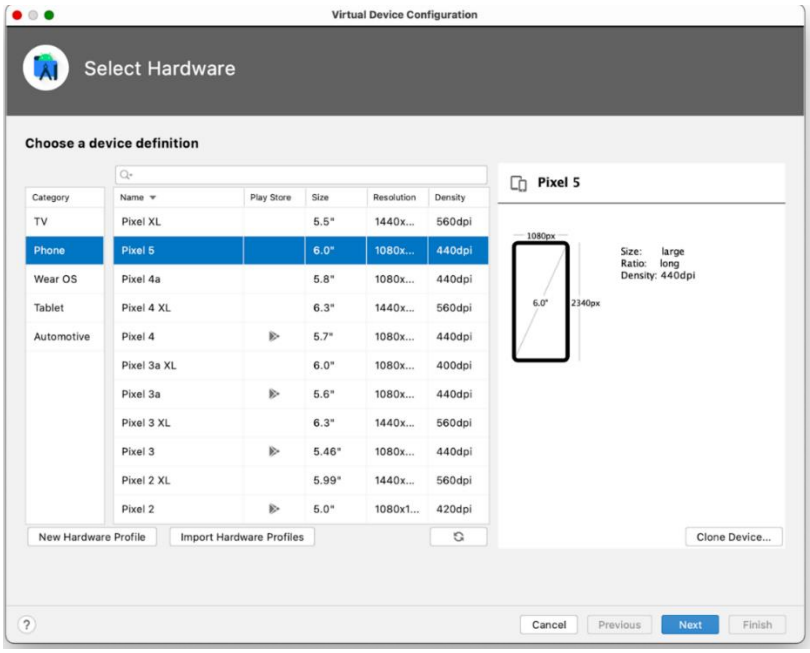
Hình 1.8: công cụ tạo thiết bị ảo

Hộp thoại **Device Manager** (Trình quản lý thiết bị) sẽ mở ra.



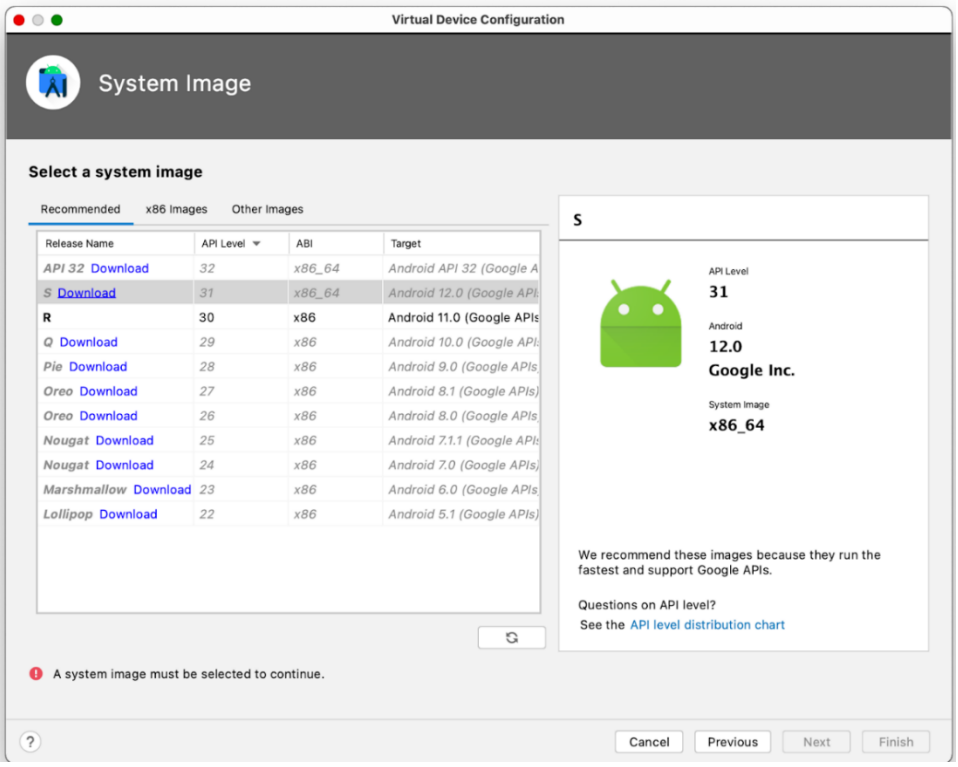
Hình 1.9: Trình quản lý thiết bị

Nhấp vào **Create virtual device** (Tạo thiết bị ảo). Hộp thoại **Virtual Device Configuration** (Cấu hình thiết bị ảo) sẽ xuất hiện.



Hình 1.10: Cửa sổ lựa chọn thiết bị

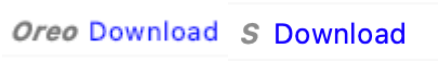
Hộp thoại này hiển thị danh sách các thiết bị đã định cấu hình trước, được sắp xếp theo danh mục để có thể chọn. Bảng này bao gồm các cột cho biết kích thước hiển Chọn danh mục **Phone** (Điện thoại).



Hình 1.11: Chọn phiên bản Android cho điện thoải ảo

Chọn một điện thoại, chẳng hạn như Pixel 5, sau đó nhấp vào **Next** (Tiếp theo). Bước này sẽ mở ra một màn hình khác. Chọn phiên bản Android để chạy trên thiết bị ảo của mình. Thao tác này cho phép thử nghiệm ứng dụng trên nhiều phiên bản Android.

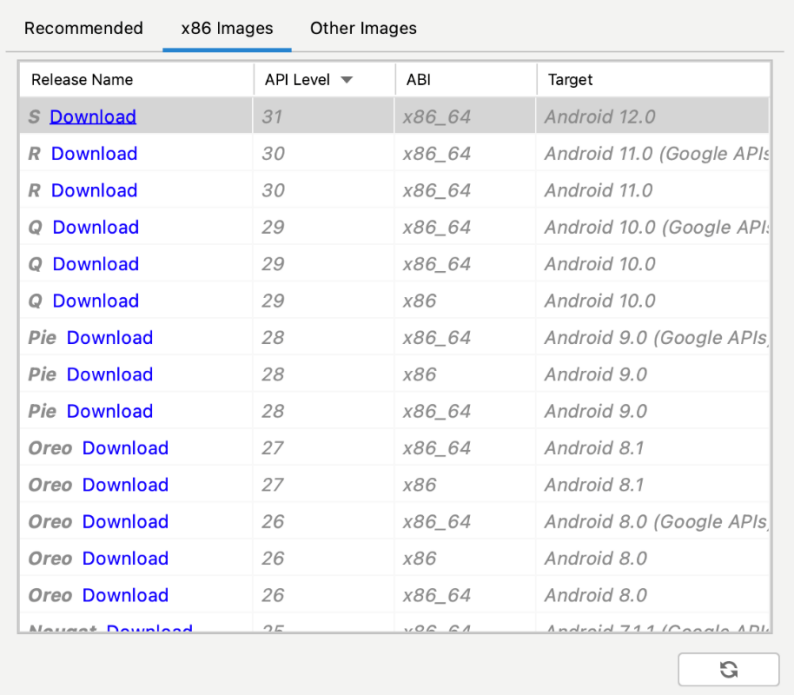
Nếu có một đường liên kết tải xuống bên cạnh **S**, hãy nhấp vào **Download** > **Accept** > **Next** > **Finish** (Tải xuống > Chấp nhận > Tiếp theo > Hoàn tất). Nếu đường liên kết tải xuống này xuất hiện, tức là hình ảnh hệ thống (system image) chưa được cài đặt trên máy tính. Trong trường hợp này, ta phải cài đặt hình ảnh hệ thống thì mới có thể định cấu hình thiết bị ảo. Quá trình tải xuống sẽ mất một chút thời gian để hoàn tất.



Hình 1.12 System image S

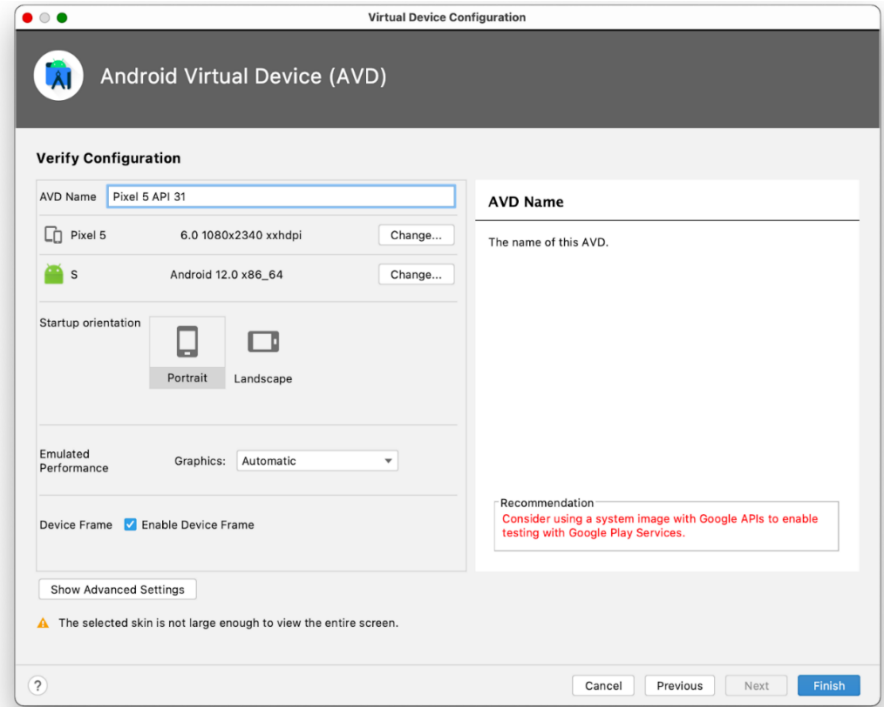
Trong thẻ **Recommended** (Đề xuất), hãy chọn S làm phiên bản Android để chạy trên thiết bị ảo.

Lưu ý quan trọng: Những ảnh hệ thống Android này chiếm nhiều dung lượng ổ đĩa. Vì vậy, chỉ một số ảnh có trong quá trình cài đặt ban đầu. Có nhiều phiên bản hệ thống Android hơn so với nội dung trong thẻ **Recommended** (Đề xuất). Để xem các phiên bản này, hãy tìm ở phần bên dưới thẻ **x86 Images** (Hình ảnh x86) và **Other Images** (Các hình ảnh khác) trong hộp thoại **Virtual Device Configuration** (Cấu hình thiết bị ảo).



Hình 1.13: Các hình ảnh hệ thống đề xuất

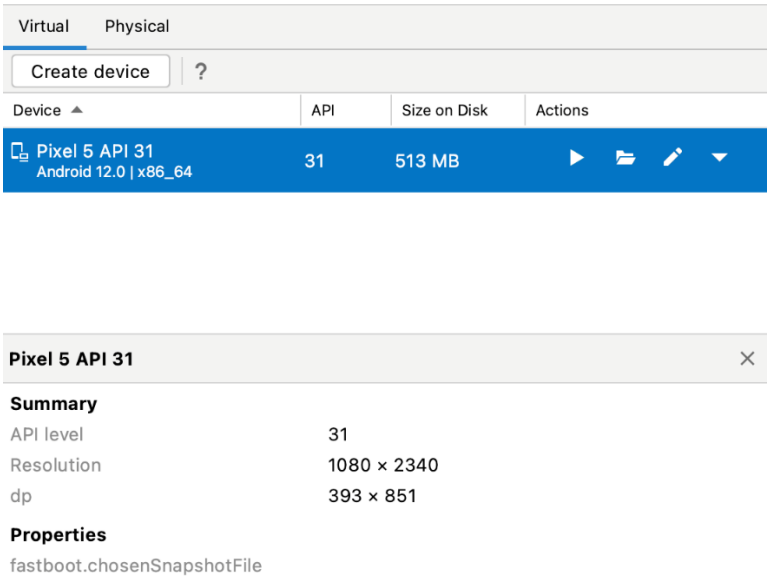
Thao tác này sẽ mở ra một màn hình khác. Chọn các thông tin bổ sung về cấu hình cho thiết bị.



Hình 1.14: Thông tin bổ sung cho AVD

Lưu ý: Nếu thấy cảnh báo màu đỏ khuyên dùng hình ảnh có Google API như trong ảnh chụp màn hình ở trên, ta có thể bỏ qua cảnh báo này.

Trong trường **AVD Name** (Tên AVD), hãy nhập tên cho AVD hoặc dùng tên mặc định. Giữ nguyên các trường còn lại. Nhấp vào **Finish** (Hoàn tất).

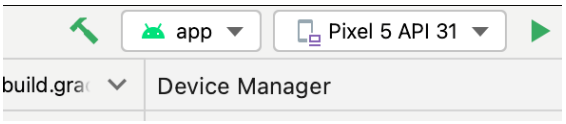


Hình 1.15: Hoàn tất cài đặt AVD

Đóng hộp thoại **Device Manager** (Trình quản lý thiết bị).

Chạy ứng dụng trên Trình mô phỏng Android

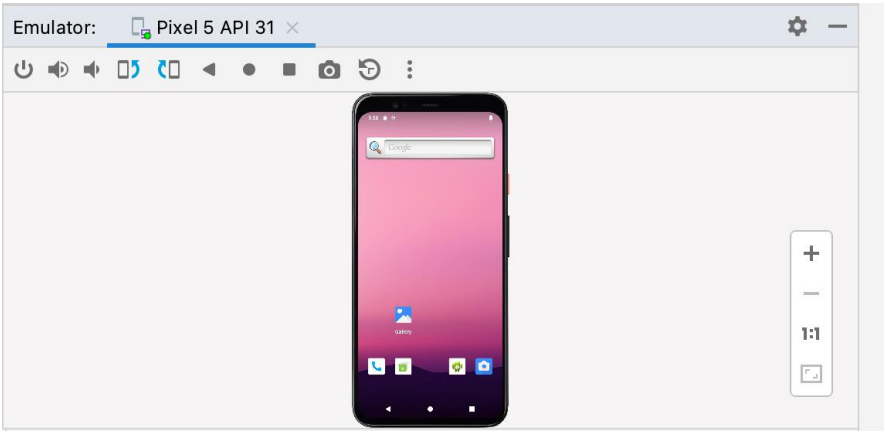
Chọn thiết bị ảo đã tạo từ trình đơn thả xuống ở đầu cửa sổ Android Studio.



Hình 1.16: Chạy ứng dụng trên AVD

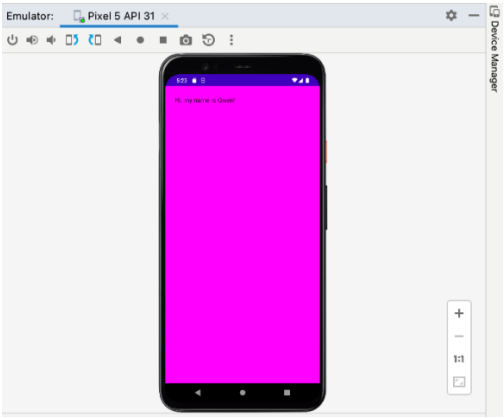
Nhấp vào nút .

Thiết bị ảo này sẽ khởi động giống như một thiết bị thực tế. Theo dự kiến, trình mô phỏng sẽ mất một chút thời gian (có thể vài phút) để khởi động lần đầu tiên. Thiết bị ảo sẽ mở ra bên cạnh trình soạn thảo mã.



Hình 1.17: Hình ảnh thiết bị AVD khi chạy

Khi đã sẵn sàng, ứng dụng sẽ mở trên thiết bị ảo.

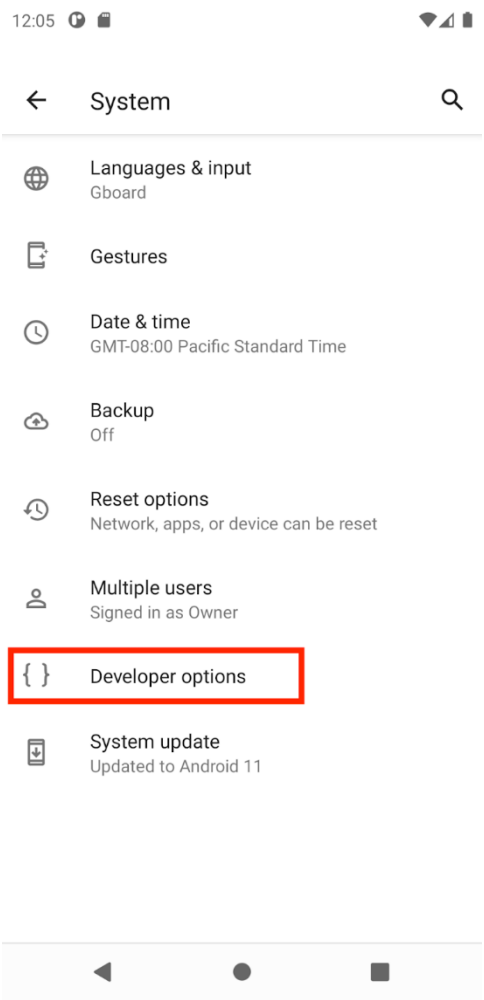


Hình 1.18: Ứng dụng chạy trên AVD

Bật tính năng gỡ lỗi USB

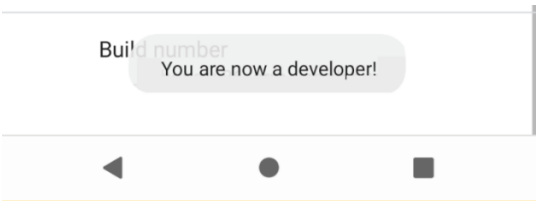
Để cho phép Android Studio giao tiếp với thiết bị Android, ta phải bật tính năng gỡ lỗi qua USB trong phần cài đặt Tuỳ chọn cho nhà phát triển của thiết bị.

Trên thiết bị Android, hãy nhấn vào **Cài đặt** > **Giới thiệu về điện thoại**. Nhấn vào **Số bản dựng** 7 lần.



Hình 1.19: Tính năng dành cho nhà phát triển

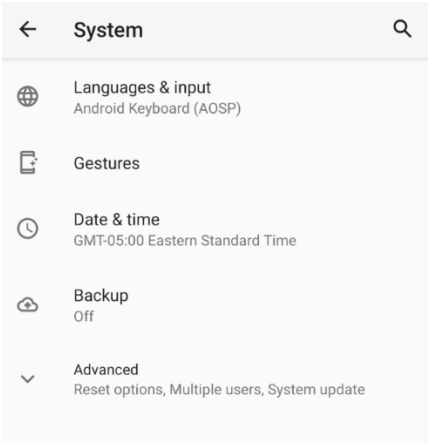
Nếu được nhắc, hãy nhập mã PIN hoặc mật khẩu của thiết bị. Khi nhìn thấy thông báo **đã là nhà phát triển!**, thì nghĩa là đã thành công.



Hình 1.20: Sau khi chọn chế độ cho nhà phát triển

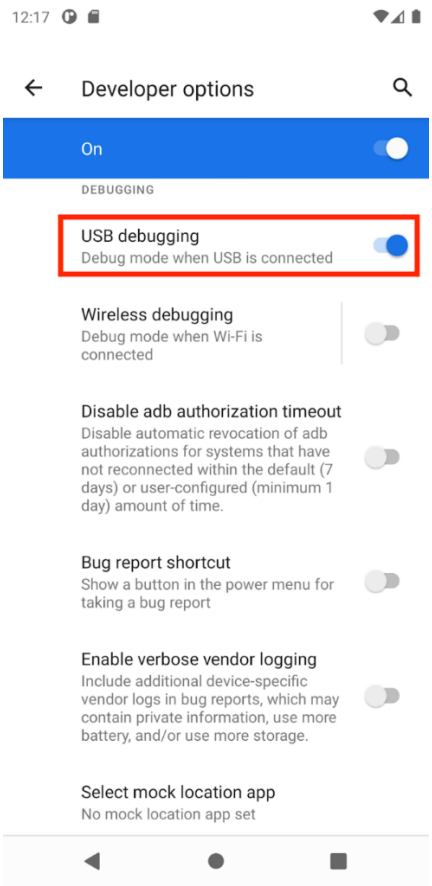
Quay lại phần **Cài đặt** rồi nhấn vào **Hệ thống** > **Tuỳ chọn cho nhà phát triển**.

Nếu không thấy **Tuỳ chọn cho nhà phát triển**, hãy nhấn vào **Tuỳ chọn nâng cao**.



Hình 1.21: Tùy chọn nâng cao

Nhấn vào **Tuỳ chọn cho nhà phát triển**, sau đó nhấn vào nút bật/tắt **Gỡ lỗi qua USB** để bật tính năng này.



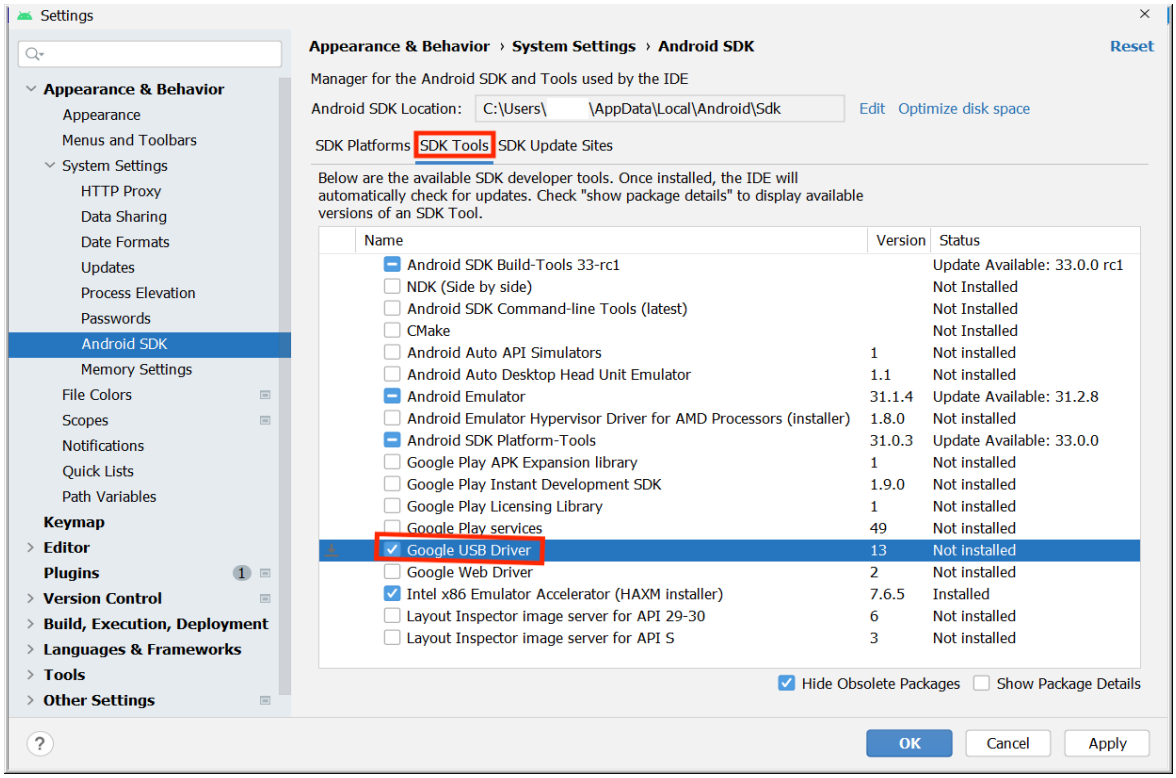
Hình 1.22: Gỡ lỗi qua USB

Cài đặt Trình điều khiển USB của Google (chỉ dành cho Windows)

Nếu đã cài đặt Android Studio trên Windows, ta cần phải cài đặt trình điều khiển thiết bị USB để có thể chạy ứng dụng của mình trên một thiết bị thực.

Trong Android Studio, hãy nhấp vào **Tools > SDK Manager** (Công cụ > Trình quản lý SDK). Hộp thoại **Preferences > Appearance & Behavior > System Settings > Android SDK** (Lựa chọn ưu tiên > Giao diện và hành vi > Cài đặt hệ thống > SDK Android) sẽ mở ra.

Nhấp vào thẻ **SDK Tools** (Bộ công cụ SDK). Chọn **Google USB Driver** (Trình điều khiển USB của Google), sau đó nhấp vào **OK**.



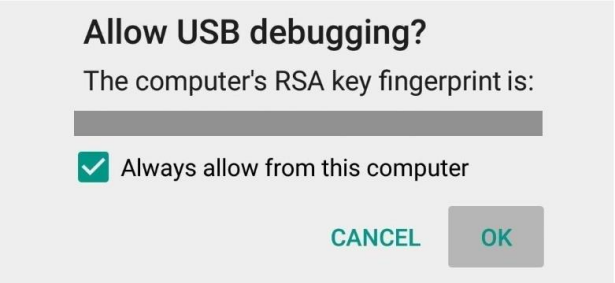
Hình 1.23: Google USB Driver

Khi hoàn tất, các tệp của trình điều khiển sẽ được tải xuống thư mục `android_sdk\extras\google\usb_driver`. Giờ đây, ta có thể kết nối và chạy ứng dụng trong Android Studio.

Chạy ứng dụng trên thiết bị Android bằng cáp

Có 2 cách để kết nối thiết bị với Android Studio: qua cáp hoặc qua Wi-Fi.

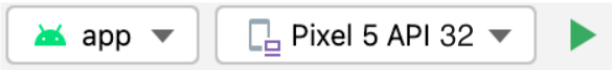
Kết nối thiết bị Android với máy tính bằng cáp USB. Một hộp thoại sẽ hiện ra trên thiết bị để yêu cầu cho phép gỡ lỗi qua USB.



Hình 1.24: Cho phép debug qua USB

Chọn hộp đánh dấu **Luôn cho phép từ máy tính này**, sau đó nhấn vào **OK**.

Trong Android Studio trên máy tính, hãy nhớ chọn thiết bị của trong trình đơn thả xuống. Nhấp vào .



Hình 1.25: Chọn thiết bị chạy demo

Chọn thiết bị rồi nhấp **OK**. Android Studio sẽ cài đặt và chạy ứng dụng đó trên thiết bị.

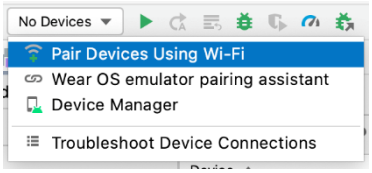
Lưu ý: Đối với Android Studio 3.6 trở lên, thiết bị thực sẽ tự động được chọn khi kết nối ở trạng thái có bật tính năng gỡ lỗi.

Nếu thiết bị chạy một nền tảng Android chưa được cài đặt trong Android Studio và có một thông báo hỏi có muốn cài đặt nền tảng cần thiết hay không, hãy nhấp vào **Install** > **Continue** > **Finish** (Cài đặt > Tiếp tục > Hoàn tất). Android Studio sẽ cài đặt và chạy ứng dụng đó trên thiết bị.

Chạy ứng dụng trên thiết bị Android bằng Wi-Fi

Ghép nối thiết bị

Trong Android Studio, hãy chọn **Pair Devices Using Wi-Fi** (Ghép nối thiết bị bằng Wi-Fi) từ trình đơn thả xuống về cấu hình chạy.

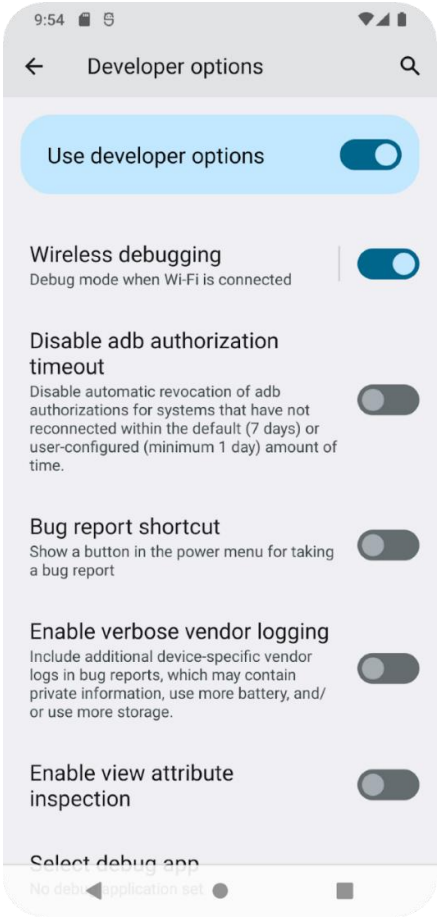


Hình 1.26 Ghép đôi thiết bị

Hộp thoại **Pair devices over Wi-Fi** (Ghép nối thiết bị qua Wi-Fi) sẽ mở ra.

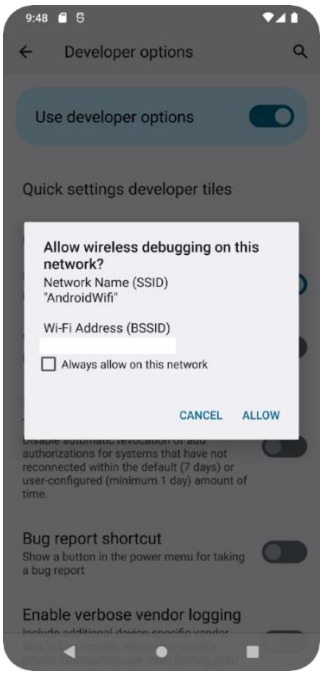
Hình 1.27: Quét mã QR

Chuyển đến **Developer options** (Tuỳ chọn cho nhà phát triển), cuộn xuống phần **Debugging** (Gỡ lỗi) rồi bật tuỳ chọn **Wireless debugging** (Gỡ lỗi qua Wi-Fi).



Hình 1.28: Gỡ lỗi qua Wifi

Trong cửa sổ bật lên **Cho phép gỡ lỗi qua Wi-Fi trên mạng này?**, hãy chọn **Cho phép**.



Hình 1.29: Chp phép gỡ lỗi thông qua mạng wifi

Nếu ghép nối thiết bị bằng mã QR, hãy chọn **Ghép nối thiết bị bằng mã QR** rồi quét mã QR trên máy tính. Ngoài ra, nếu ghép nối thiết bị bằng mã ghép nối, hãy chọn **Ghép nối thiết bị bằng mã ghép nối**, sau đó nhập mã gồm 6 chữ số

1.2. Lập trình cơ bản với ngôn ngữ Kotlin (3 tiết)

Hình 1.30 Kotlin

Kotlin là ngôn ngữ lập trình hiện đại, được JetBrains phát hành chính thức vào năm 2011. Kotlin 1.0, ra mắt vào ngày 15 tháng 2 năm 2016, được công nhận là bản phát hành ổn định đầu tiên. Bắt đầu với phiên bản này, JetBrains đã cam kết khả năng tương thích ngược lâu dài. Kotlin được thiết kế để khắc phục một số nhược điểm của Java, chẳng hạn như tính dài dòng và ngoại lệ con trỏ null. Những người tạo ra Kotlin nhằm mục đích phát triển một ngôn ngữ có thể sử dụng các khung và thư viện Java hiện có đồng thời cung cấp cú pháp rõ ràng và biểu cảm hơn.

Kotlin tự hình dung mình là một ngôn ngữ thực dụng, cung cấp giải pháp cho những thách thức lập trình phổ biến. Nó hoàn toàn có thể tương tác với Java, điều đó có

nghĩa là nó có thể sử dụng tất cả các thư viện, khung công tác và công cụ Java hiện có. Ngoài ra, có thể gọi mã Kotlin từ Java và ngược lại.

Những đặc điểm chính:

- Tính chính xác: Kotlin giảm thiểu mã soạn sẵn, dẫn đến cần ít dòng mã hơn để đạt được cùng chức năng. Ví dụ: các lớp dữ liệu trong Kotlin sẽ tự động tạo equals() các phương thức hashCode()và toString():

```
data class User(val name: String, val age: Int)
```

- Tính biểu cảm: Cú pháp của Kotlin cho phép lập trình viên viết mã dễ đọc và dễ hiểu hơn. Ví dụ: when biểu thức là phiên bản mạnh hơn của Java switch:

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> print("x is neither 1 nor 2")
}
```

- Khả năng tương tác với Java: Kotlin được thiết kế để hoàn toàn có thể tương tác với Java. Tính năng này có nghĩa là có thể dễ dàng gọi mã Kotlin từ Java hoặc mã Java từ Kotlin:

```
// Kotlin function
fun getList(): List<String> = listOf("Kotlin", "Java")
// Java code calling the Kotlin function
List<String> list = MyKotlinFile.getList();
```

Kotlin nhanh chóng trở nên phổ biến và vào năm 2017, Google đã công bố nó là ngôn ngữ phát triển Android chính thức. Các tính năng hiện đại và khả năng tích hợp liền mạch với Java khiến nó trở thành lựa chọn hấp dẫn cho cả dự án mới và dự án hiện có.

Thiết lập môi trường phát triển

Để bắt đầu viết mã bằng Kotlin, cần thiết lập môi trường phát triển của mình. Có thể sử dụng Kotlin để phát triển Android, ứng dụng phụ trợ và ứng dụng dành cho máy tính để bàn. Sau đây là hướng dẫn đơn giản để cài đặt Kotlin và thiết lập IDE:

- Cài đặt Bộ công cụ phát triển Java (JDK): Đảm bảo đã cài đặt JDK vì Kotlin chạy trên Máy ảo Java (JVM). Có thể tải xuống từ trang web chính thức của Oracle hoặc sử dụng OpenJDK.

- Cài đặt IntelliJ IDEA, Android Studio hoặc Eclipse: Để phát triển chung, IntelliJ IDEA Community Edition miễn phí và hỗ trợ Kotlin ngay từ đầu. Có thể tải xuống từ trang web JetBrains. Để phát triển Android, Android Studio, được xây dựng trên IntelliJ, là lựa chọn tốt nhất. Nó có sẵn trên trang web

của nhà phát triển Android. Eclipse có thể được sử dụng để thay thế cho IntelliJ, tuy nhiên IntelliJ cung cấp chất lượng tùy chỉnh và trải nghiệm người dùng cao hơn. Trong quá trình cài đặt, hãy nhớ chọn plugin Kotlin nếu plugin này không được bao gồm theo mặc định.

- Định cấu hình IDE:
  - Mở IDE của và tạo một dự án mới.
  - Chọn Kotlin làm ngôn ngữ và chọn loại dự án thích hợp (JVM/Android).
  - Định cấu hình SDK dự án cho JDK mà đã cài đặt.
- Sử dụng Kotlin REPL: Kotlin REPL (Vòng lặp đọc-Eval-Print) là một công cụ giúp chạy nhanh các đoạn mã Kotlin mà không cần tạo một dự án đầy đủ. Thật tuyệt vời để thử nghiệm. Truy cập nó trong IntelliJ IDEA bằng cách điều hướng đến Tools -> Kotlin -> Kotlin REPL.

Đây là đoạn mã cơ bản để thử trong REPL:

```
fun main() {
    val greeting = "Hello, Kotlin!"
    println(greeting)
}
```

Sao chép và dán phần này vào REPL và nó sẽ thực thi, in "Xin chào, Kotlin!" đến bảng điều khiển.

Bằng cách làm theo các bước này, ta sẽ có một môi trường phát triển Kotlin đầy đủ chức năng, sẵn sàng để xây dựng ứng dụng hoặc thử nghiệm ngôn ngữ.

Cú pháp và khái niệm cơ bản trong Kotlin

Kotlin là ngôn ngữ lập trình kiểu tĩnh chạy trên Máy ảo Java (JVM). Nó được thiết kế để hoàn toàn có thể tương tác với Java trong khi cung cấp cú pháp ngắn gọn hơn và các tính năng được cải tiến.

Biến

Trong Kotlin, sử dụng **val** từ khóa để khai báo các biến bất biến và **var** các biến có thể thay đổi.

```
val immutableString: String = "Hello" // Immutable variable
var mutableInt: Int = 42 // Mutable variable
```

Loại dữ liệu

Kotlin đi kèm với các kiểu dữ liệu tích hợp sẵn như Int, Double, Float, Booleanv.v. Nó tự động suy ra loại biến nếu không được nêu rõ ràng.

```
val inferredType = 42 // Int inferred
```

Toán tử

Kotlin sử dụng các toán tử tương tự như Java, như +, -, *, /và %. Nó cũng hỗ trợ quá tải toán tử .

Kiểm soát dòng chảy

Trong Kotlin, có thể sử dụng câu if lệnh làm biểu thức, đây là một cải tiến so với cách sử dụng truyền thống của Java.

```
val max = if (a > b) a else b
```

Vòng lặp trong Kotlin tương tự như Java nhưng có cải tiến. Vòng for lặp mạnh hơn với ..toán tử for range .

```
for (i in 1..10) {
    println(i)
}
```

Các vòng lặp while và do..while hoạt động tương tự như trong Java.

Kotlin giới thiệu when biểu thức, một dạng nâng cao của biểu thức chuyển đổi, hỗ trợ các biểu thức tùy ý.

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> { // Note the block
        print("x is neither 1 nor 2")
    }
}
```

Những cải tiến trên Java

Cú pháp của Kotlin ngắn gọn hơn, giảm bớt mã nguyên mẫu. Ví dụ: không cần sử dụng dấu chấm phẩy ở cuối mỗi câu lệnh. Suy luận kiểu làm giảm tính dài dòng của việc khai báo biến. Biểu thức này when linh hoạt hơn trường hợp chuyển đổi của Java và Kotlin if có thể được sử dụng làm biểu thức, loại bỏ sự cần thiết của toán tử ba ngôi.

Kotlin tập trung vào sự an toàn với các loại có thể rỗng và các tính năng chức năng của nó khiến Kotlin trở thành một lựa chọn mạnh mẽ để phát triển ứng dụng hiện đại.

Công cụ sử dụng:

https://developer.android.com/training/kotlinplayground?hl=vi

https://play.kotlinlang.org/koans/Introduction/Hello,%20world!/Task.kt

1.2.1. Toán tử trong Kotlin

- Mathematical operators `+ - * / %`
- Increment and decrement operators `++ --`
- Comparison operators `< <= > >=`
- Assignment operator `=`
- Equality operators `== !=`

1.2.2. Kiểu dữ liệu cơ bản

Kotlin đi kèm với các kiểu dữ liệu tích hợp sẵn như Int, Double, Float, Boolean v.v. Nó tự động suy ra loại biến nếu không được nêu rõ ràng.

```
val inferredType = 42 // Int inferred
```

1.2.3. Biến và khai báo biến

Trong Kotlin, sử dụng val từ khóa để khai báo các biến bất biến và var các biến có thể thay đổi.

```
val immutableString: String = "Hello" // Immutable variable
var mutableInt: Int = 42 // Mutable variable
```

1.2.4. Cấu trúc điều khiển trong Kotlin

Đây là các cú pháp điều khiển cơ bản trong Kotlin, giúp người lập trình điều khiển luồng chương trình một cách hiệu quả.

. Câu lệnh if

```
if (condition) {
    // Thực hiện nếu điều kiện đúng
} else {
    // Thực hiện nếu điều kiện sai
}
```

Câu lệnh when

```
when (variable) {
  value1 -> {
    // Thực hiện nếu variable bằng value1
  }
  value2 -> {
    // Thực hiện nếu variable bằng value2
  }
  else -> {
    // Thực hiện nếu không khớp với các giá trị trên
  }
}
```

Câu lệnh for

Lặp qua một dãy số:

```
for (i in 1..10) {
  // Thực hiện cho i từ 1 đến 10
}
```

Lặp qua một danh sách:

```
val list = listOf("a", "b", "c")
for (item in list) {
  // Thực hiện cho từng item trong danh sách
}
```

Câu lệnh while

```
while (condition) {
  // Thực hiện khi điều kiện đúng
}
```

Câu lệnh do...while

```
do {
  // Thực hiện ít nhất một lần
} while (condition)
```

Câu lệnh break và continue

break: Dừng vòng lặp.

```
for (i in 1..10) {
  if (i == 5) break // Dừng vòng lặp khi i bằng 5
}
```

continue: Bỏ qua phần còn lại của vòng lặp và tiếp tục với vòng lặp tiếp theo.

```
for (i in 1..10) {
  if (i % 2 == 0) continue // Bỏ qua số chẵn
  // Thực hiện cho số lẻ
}
```

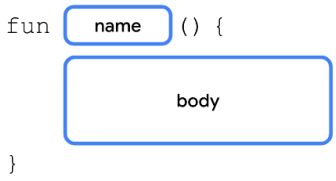
1.2.5. Hàm và cách định nghĩa hàm

Định nghĩa và gọi một hàm

Việc khai báo (hoặc định nghĩa) hàm sẽ sử dụng từ khoá `fun` và thêm mã vào dấu ngoặc nhọn.

```
fun main() {
    println("Happy Birthday, Rover!")
    println("You are now 5 years old!")
}
```

Biểu đồ sau đây thể hiện cú pháp để định nghĩa một hàm.



Hình 1.31 Định nghĩa một hàm

Định nghĩa hàm bắt đầu bằng từ khoá `fun`, theo sau là tên hàm, một cặp dấu ngoặc đơn và một cặp dấu ngoặc nhọn. Bên trong dấu ngoặc nhọn là mã sẽ chạy khi hàm được gọi. Ví dụ:

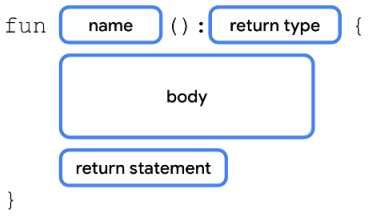
```
fun main() {
    birthdayGreeting()
}

fun birthdayGreeting() {
    println("Happy Birthday, Rover!")
    println("You are now 5 years old!")
}
```

Trả về một giá trị từ một hàm

Các hàm Kotlin cũng có thể tạo dữ liệu được gọi là giá trị trả về. Giá trị này được lưu trong một biến mà ta có thể dùng ở những nơi khác trong mã.

Khi định nghĩa một hàm, ta có thể chỉ định kiểu dữ liệu cho giá trị mà mình muốn trả về. Chỉ định kiểu dữ liệu trả về bằng cách đặt dấu hai chấm (:) sau dấu ngoặc đơn, một khoảng trống và tên kiểu (Int, String, v.v.). Sau đó, một dấu cách sẽ được đặt giữa kiểu dữ liệu trả về và dấu ngoặc nhọn mở. Trong phần nội dung của hàm, sau tất cả câu lệnh, ta sẽ sử dụng câu lệnh trả về để chỉ định giá trị mà mình muốn hàm trả về. Câu lệnh trả về bao gồm từ khoá return nằm trước giá trị (chẳng hạn như biến) mà ta muốn hàm trả về ở dạng dữ liệu đầu ra.

Dưới đây là cú pháp khai báo hàm có cùng kiểu dữ liệu trả về.



Hình 1.32 Định nghĩa một hàm đầy đủ

Kiểu Unit

Theo mặc định, nếu không chỉ định loại dữ liệu trả về, thì loại dữ liệu trả về mặc định là Unit. Unit có nghĩa là hàm không trả về một giá trị. Unit tương đương với các loại dữ liệu trả về không có giá trị ở các ngôn ngữ khác (void ở Java và C; bộ Void/dữ liệu rỗng () ở Swift; None ở Python, v.v.). Bất kỳ hàm nào không trả về một giá trị đều sẽ trả về Unit.

Trong phần khai báo hàm cho birthdayGreeting(), hãy thêm dấu hai chấm sau dấu ngoặc đơn đóng và chỉ định loại dữ liệu trả về là Unit.

```
fun main() {
    birthdayGreeting()
}

fun birthdayGreeting(): Unit {
    println("Happy Birthday, Rover!")
    println("You are now 5 years old!")
}
```

Kết quả chạy:

Happy Birthday, Rover!

You are now 5 years old!

Không bắt buộc phải chỉ định loại dữ liệu trả về Unit trong Kotlin. Đối với các hàm không trả về dữ liệu nào hoặc trả về Unit, ta không cần dùng câu lệnh trả về.

Để minh hoạ cách hàm có thể trả về một giá trị, sửa đổi hàm birthdayGreeting() để trả về một chuỗi thay vì chỉ in kết quả.

Thay thế loại trả về Unit bằng String.

```
fun birthdayGreeting(): String {
    val nameGreeting = "Happy Birthday, Rover!"
    val ageGreeting = "You are now 5 years old!"
}
```

Hãy thêm câu lệnh `return` bằng cách dùng cú pháp định dạng chuỗi để trả về một chuỗi từ hàm bao gồm cả lời chúc.

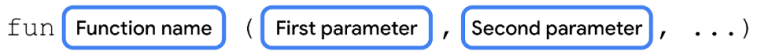
```
fun birthdayGreeting(): String {
    val nameGreeting = "Happy Birthday, Rover!"
    val ageGreeting = "You are now 5 years old!"
    return "$nameGreeting\n$ageGreeting"
}
```

Trong main(), hãy gọi println() để in chuỗi greeting. Hàm main() giờ đây phải có dạng như sau.

```
fun main() {
    val greeting = birthdayGreeting()
```

```
    println(greeting)
}
```

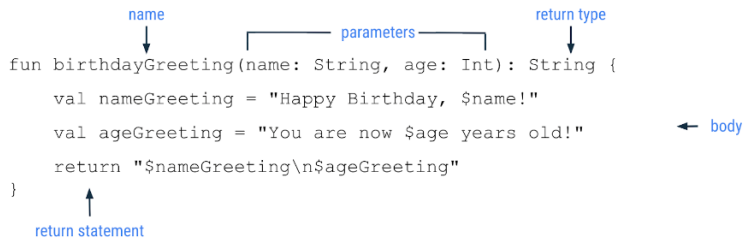
Hàm có nhiều tham số



Hình 1.33 Truyền tham số cho hàm

Các tham số đã định nghĩa được phân tách bằng dấu phẩy. Tương tự, khi gọi một hàm có nhiều tham số, ta cũng sẽ phân tách các tham số được chuyển vào bằng dấu phẩy. Hãy xem ví dụ thực tế.

```
fun birthdayGreeting(name: String, age: Int): String {
    val nameGreeting = "Happy Birthday, $name!"
    val ageGreeting = "You are now $age years old!"
    return "$nameGreeting\n$ageGreeting"
}
```



Hình 1.34 Ví dụ một hàm đầy đủ

Tham số được đặt tên

Trong các ví dụ trước, ta không cần chỉ định tên tham số, `name` hoặc `age`, khi gọi hàm. Tuy nhiên, có thể làm vậy nếu muốn khi ta bao gồm tên tham số khi gọi một hàm, hàm này được gọi là *tham số được đặt tên*.

```
println(birthdayGreeting(name = "Rex", age = 2))
```

Sắp xếp lại thứ tự các tham số được đặt tên. Ví dụ: đặt tham số được đặt tên `age` trước tham số được đặt tên `name`.

```
println(birthdayGreeting(age = 2, name = "Rex"))
```

Chạy mã và ta thấy rằng kết quả vẫn như cũ. Mặc dù đã thay đổi thứ tự của các tham số, nhưng các giá trị tương tự vẫn được chuyển vào các tham số đó.

Happy Birthday, Rover!

You are now 5 years old!

Happy Birthday, Rex!

You are now 2 years old!

Tham số mặc định

Các tham số của hàm cũng có thể chỉ định các tham số mặc định. Khi gọi một hàm, có thể chọn bỏ qua tham số mang giá trị mặc định. Trong trường hợp này, giá trị mặc định sẽ được dùng.

Để thêm tham số mặc định, hãy thêm toán tử gán (=) sau loại dữ liệu cho tham số và đặt một giá trị cho tham số này.

```
fun birthdayGreeting(name: String = "Rover", age: Int): String {
    return "Happy Birthday, $name! You are now $age years old!"
}
```

Trong lệnh gọi đầu tiên tới `birthdayGreeting()` cho Rover trong `main()`, hãy đặt tham số được đặt tên `age` thành `5`. Vì tham số `age` được định nghĩa sau `name` nên cần dùng tham số được đặt tên `age`. Nếu không có tham số có tên, Kotlin sẽ giả định thứ tự các tham số giống như thứ tự những tham số được xác định. Việc sử dụng tham số có tên là nhằm đảm bảo Kotlin nhận được `Int` cho tham số `age`.

```
println(birthdayGreeting(age = 5))
println(birthdayGreeting("Rex", 2))
```

Chạy mã. Lệnh gọi đầu tiên tới hàm `birthdayGreeting()` sẽ in tên "Rover" vì chưa chỉ định tên. Lệnh gọi thứ hai đến `birthdayGreeting()` vẫn sử dụng giá trị `Rex` chuyển cho `name`.

Happy Birthday, Rover! You are now 5 years old!

Happy Birthday, Rex! You are now 2 years old!

1.2.6. Lập trình Kotlin trong Android Studio

Tại sao Kotlin được ưa chuộng trong lập trình Android?

- **Cú pháp ngắn gọn và dễ hiểu**: Kotlin giúp giảm thiểu mã nguồn dài dòng và phức tạp, làm cho mã nguồn dễ bảo trì hơn.
- **Tính năng Null Safety**: Kotlin loại bỏ lỗi NullPointerException thông qua hệ thống kiểu dữ liệu an toàn với null.
- **Kết hợp với Java**: Kotlin hoàn toàn tương thích với Java, do đó có thể tích hợp vào các dự án Android hiện có mà không cần thay đổi toàn bộ mã nguồn.

- **Hỗ trợ tính năng hiện đại**: Các tính năng như Extension functions, Lambda expressions, Data classes giúp việc lập trình trở nên dễ dàng và trực quan hơn.

Kotlin và Android: Tích Hợp Cùng Android Studio

- **Android Studio** là IDE chính thức của Google để phát triển ứng dụng Android. Kotlin được hỗ trợ trực tiếp trong Android Studio, giúp lập trình viên dễ dàng viết, kiểm tra và triển khai ứng dụng Android bằng Kotlin.
- Khi khởi tạo một dự án Android mới, có thể chọn Kotlin là ngôn ngữ chính thay cho Java. Android Studio sẽ tự động cấu hình và hỗ trợ đầy đủ tất cả các công cụ cần thiết cho lập trình Kotlin.

Cách Sử Dụng Kotlin Trong Android

Các Thành Phần Cơ Bản:

- Activity và Fragment: Cũng giống như Java, trong Kotlin sẽ viết các class Activity và Fragment để quản lý giao diện người dùng của ứng dụng Android.
- View và ViewGroup: Sử dụng các lớp TextView, Button, LinearLayout, ConstraintLayout... giống như trong Java nhưng với cú pháp Kotlin ngắn gọn hơn.

Xử Lý Sự Kiện:

- Cách sử dụng sự kiện trong Kotlin rất đơn giản và trực quan. Ví dụ, để xử lý sự kiện click của một nút Button, chỉ cần sử dụng cú pháp ngắn gọn sau:

```
val btnClickMe: Button = findViewById(R.id.btnClickMe)
btnClickMe.setOnClickListener {
    // Xử lý sự kiện
}
```

Null Safety:

- Kotlin giúp xử lý các lỗi null pointer bằng cách cho phép chỉ định rõ ràng các biến có thể hoặc không thể nhận giá trị null. Điều này giúp tránh được nhiều lỗi runtime phổ biến. Ví dụ:

```
var name: String? = null  // Biến có thể nhận giá trị null
name?.let { println(it) }  // Kiểm tra và xử lý nếu không null
```

Những Tính Năng Mới Được Hỗ Trợ Trong Kotlin cho Android

- Extension Functions: Kotlin cho phép mở rộng các lớp có sẵn mà không cần phải kế thừa chúng. Điều này rất hữu ích khi thêm các phương thức cho các lớp Android mà không cần thay đổi mã nguồn gốc.

```
fun Button.setTextAndColor(text: String, color: Int) {
    this.text = text
    this.setTextColor(color)
}
```

- Data Classes: Kotlin cung cấp data class giúp dễ dàng tạo các đối tượng chỉ chứa dữ liệu mà không cần phải viết nhiều mã boilerplate như toString(), equals(), hay hashCode().

```
data class User(val name: String, val age: Int)
```

- Coroutine: Kotlin có khả năng xử lý tác vụ bất đồng bộ với coroutine, giúp thực thi các thao tác như tải dữ liệu từ API mà không làm gián đoạn giao diện người dùng (UI).

```
GlobalScope.launch {
    val result = fetchDataFromNetwork()
    // Cập nhật UI trên thread chính
    withContext(Dispatchers.Main) {
        textView.text = result
    }
}
```

Lợi Ích Khi Dùng Kotlin Cho Android

- **Giảm Thiểu Lỗi**: Kotlin giúp tránh các lỗi phổ biến trong Java như NullPointerException và giảm thiểu mã nguồn không cần thiết.
- **Cộng Đồng và Hỗ Trợ Tốt**: Kotlin đã được cộng đồng Android chấp nhận và sử dụng rộng rãi, vì vậy sẽ dễ dàng tìm kiếm tài liệu và giải pháp cho các vấn đề gặp phải.
- **Hiệu Quả**: Viết mã bằng Kotlin thường nhanh chóng hơn và dễ bảo trì hơn, đặc biệt khi ứng dụng của trở nên phức tạp.

Các Tính Năng Kotlin Quan Trọng Được Sử Dụng Trong Android

- **Lambda Expressions**: Kotlin hỗ trợ các biểu thức lambda, giúp xử lý các sự kiện như nhấn nút hay các callback dễ dàng và ngắn gọn.
- **Sealed Classes**: Các lớp sealed giúp dễ dàng làm việc với các trạng thái khác nhau trong ứng dụng, ví dụ như khi xử lý các lỗi hoặc sự kiện từ API.

Bài tập bổ trợ

Bài tập 1: Khám phá môi trường phát triển Android Studio

Hướng dẫn:

- Cài đặt Android Studio:
- Tải và cài đặt Android Studio từ link chính thức.
- Cấu hình thiết bị ảo (AVD) bằng cách chọn Tools > Device Manager > Create Virtual Device.
- Tạo dự án HelloMobileApp:
- Mở Android Studio, chọn New Project > Empty Activity.
- Đặt tên dự án là HelloMobileApp.
- Chạy ứng dụng trên thiết bị ảo và xác nhận màn hình hiển thị Hello World!.
- Thay đổi tên ứng dụng:
- Mở file res/values/strings.xml, thay đổi nội dung:
- <string name="app_name">Ứng dụng đầu tiên</string>
- Chạy lại ứng dụng trên thiết bị ảo để kiểm tra.

Bài tập 2: Lập trình cơ bản với Kotlin

1. Tính tổng các số từ 1 – n
2. Kiểm tra một số có nguyên tố không?
3. Kiểm tra một số có phải số đối xứng không?
4. Kiểm tra một số có phải là Strong Number không?

5. In ra số lẻ từ danh sách:

Bài giải:

```
fun main() {
    val numbers = (1..10).toList()
    println("Các số lẻ trong danh sách:")
    for (number in numbers) {
        if (number % 2 != 0) {
            println(number)
```

```
        }
    }
}
```

Kết quả chạy:

```
Các số lẻ trong danh sách:
1
3
5
7
9
```

Hàm tinhTong:

```
fun tinhTong(a: Int, b: Int): Int {
    return a + b
}

fun main() {
    val tong = tinhTong(5, 7)
    println("Tổng của 5 và 7 là: $tong")
}
```

Kết quả chạy:

```
Tổng của 5 và 7 là: 12
```